

Design and Analysis of Algorithm (DAA), Lab-02

BTech (CS-B and CE), 3rd Semester
August 4, 2026



Instructor: Dr. Ajaya Kumar Dash.

Questions

1. **Dictionary Operations:** Consider a dictionary (D) abstract data type that permits access to data items by content (key). The following are the primary operations that the dictionary supports.
 - $Search(D, k)$ - Given a search key k , return a pointer to the elements in the dictionary D whose key value is k , if one exists.
 - $Insert(D, x)$ - Given a data item x , add it to the Dictionary D .
 - $Delete(D, x)$ - Given a pointer x to a data item in the dictionary D , remove it from D .
 - $Max(D)$ or $Min(D)$ - Retrieve the item with the largest (or smallest) key from D . These two operations help the dictionary to serve as a priority queue.
 - $Predecessor(D, x)$ or $Successor(D, x)$ Retrieve the item from D whose key is immediately before (or after) the item x in sorted order.

What are the asymptotic worst-case running times for all the seven primary operations (search, insert, delete, maximum, minimum, predecessor, and successor) when the said data structure is implemented using:

- An unsorted array.
- A sorted array.
- A singly linked unsorted list.
- A singly linked sorted list.
- A doubly linked unsorted list.
- A doubly linked sorted list.

Write a C program to validate your claim by plotting the order of growth of these functions.

2. **Merge sort vs. modified merge sort:** Consider the following modification to merge sort: divide the input array into thirds (rather than halves), recursively sort each third, and finally combine the results using a three-way merge subroutine. What is the worst-case running time of this modified merge sort?

Write a C program to validate your claim by plotting the order of growth for both the merge sort (discussed in the class) and the modified merge sort described above.

3. **Merging k sorted arrays:** Suppose you are given k sorted arrays, each with n elements, and you want to combine them into a single sorted array of kn elements.
- (a) **[Method 1]** One approach is to use the merge subroutine repeatedly, merging the first two arrays, then merging the result with the third array, then with the fourth array, and so on until you merge in the k^{th} and final input array. Write a program to validate this idea. Also, find the worst-case running time of this method.
 - (b) **[Method 2]** Consider the algorithm that first divides the k arrays into $\frac{k}{2}$ pairs of arrays and uses the merge subroutine to combine each pair, resulting in $\frac{k}{2}$ sorted-length $2n$ arrays. The algorithm repeats this step until there is only one length- kn sorted array. Write a program to validate this idea. What is the running time of this method as a function of n and k ?
-